

CLIPS EXPERT SYSTEMS SHELL

BITI 2213 - Lab

Table of Contents

1. What is CLIPS?
2. CLIPS tutorials
 - i. Facts and Rules
 - ii. Pattern Matching
 - iii. Wildcards and Variables
 - iv. Templates and Conditions
 - v. Truth Maintenance and Conflict Resolution
 - vi. Others

What is CLIPS?

- CLIPS is a forward–chaining, pattern-matching knowledge-based system shell.
- CLIPS stands for *C* Language *I*mplementation *P*roduction *S*ystem.
- CLIPS was developed by the Artificial Intelligence Section of the Johnson Space Center of NASA.
- Its pattern-matching algorithm is based on the Rete algorithm.
- CLIPS provides high portability, low cost, and high degree of integration with external programs.

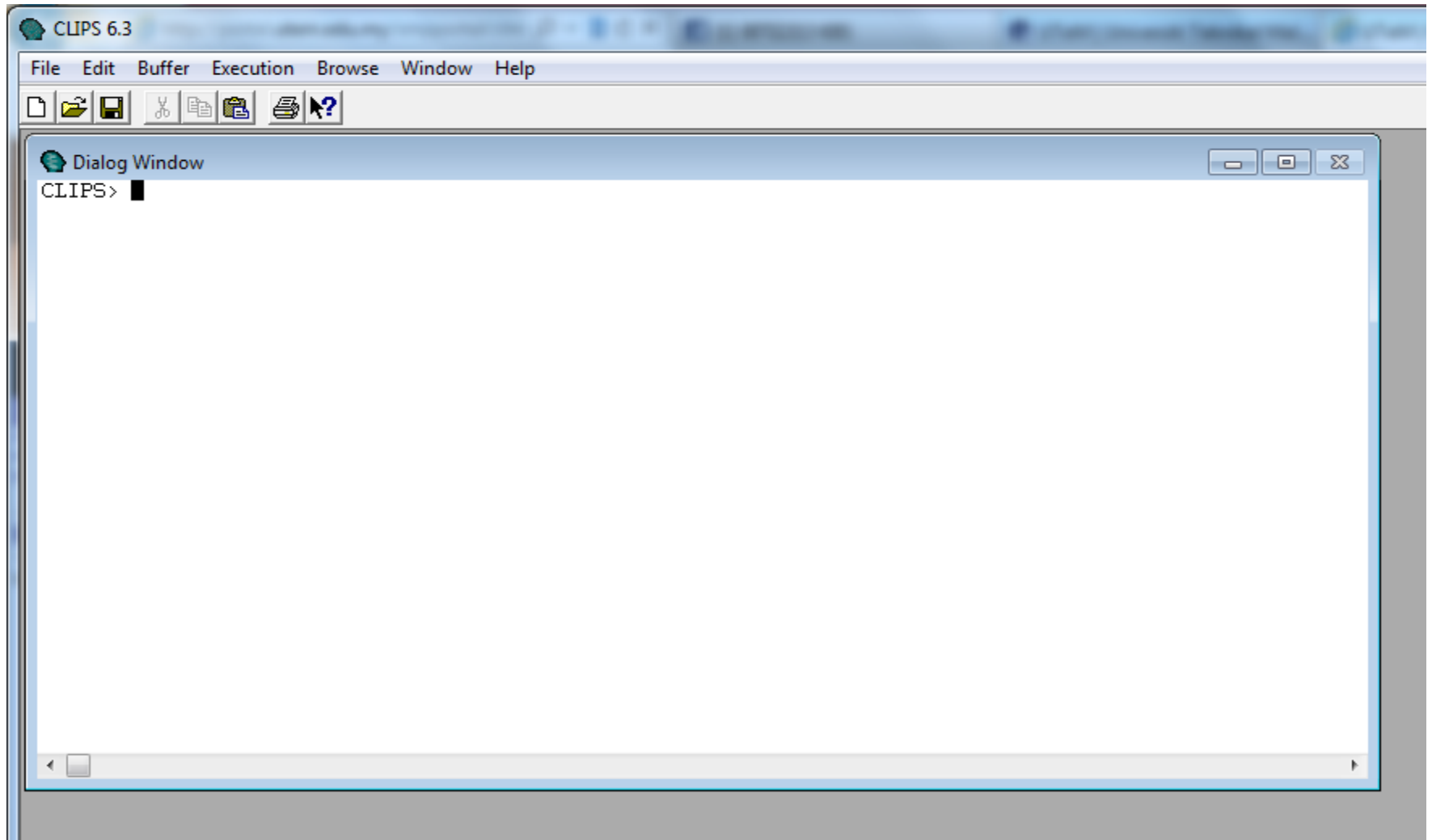
CLIPS tutorials

- The following six short tutorials will step you through most of the basic information you need in order to use the CLIPS expert system development tool. The tutorials were developed by Peter Robinson at the University of Hull in Great Britain.
- Topic 1: Facts and Rules
- Topic 2: Pattern Matching
- Topic 3: Wildcards and Variables
- Topic 4: Templates and Conditions
- Topic 5: Truth Maintenance and Conflict Resolution
- Topic 6: Other Stuff

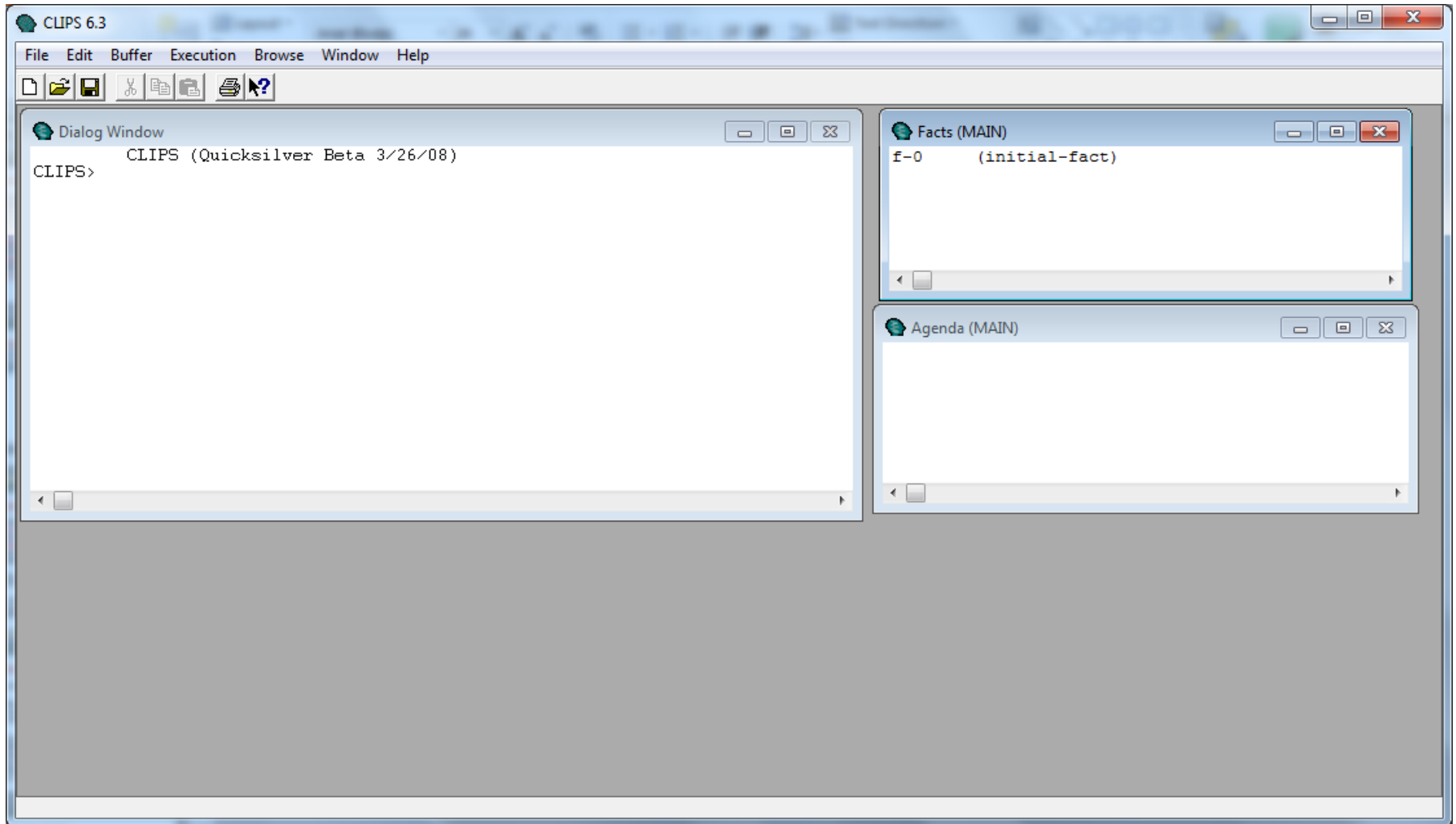
Introduction

- If CLIPS isn't installed on your computer, download CLIPS installer from Ulearn-BITI2213.
- Double-click on the installer to install CLIPS.
- To start CLIPS, double-click on the CLIPSWin.exe file.
- (exit) to shut down CLIPS
- (clear) to remove all rules and facts from memory. Equivalent to shutting down and restarting CLIPS.
- (reset) to remove facts from memory and resets the agenda.
- (run) to start executing a CLIPS program.

Introduction: First Screen of CLIPS

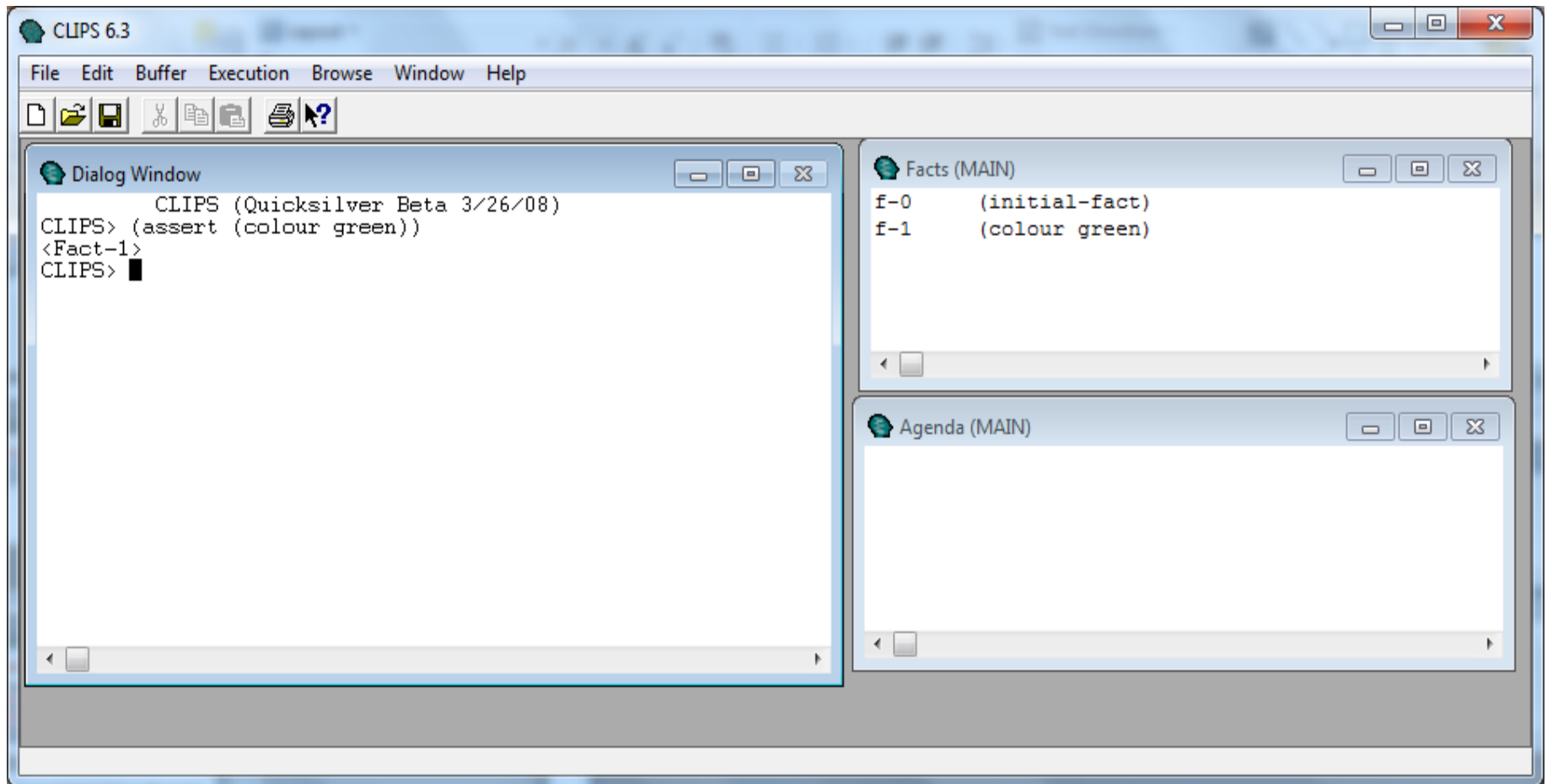


Introduction: Screen of CLIPS



- Click Window menu and check Fact (MAIN) and Agenda (MAIN).

Introduction: Screen of CLIPS



- Check the Fact (MAIN) window. What has been added into?

CLIPS tutorials

- The following six short tutorials will step you through most of the basic information you need in order to use the CLIPS expert system development tool. The tutorials were developed by Peter Robinson at the University of Hull in Great Britain.

Topic 1: Facts and Rules

Topic 2: Pattern Matching

Topic 3: Wildcards and Variables

Topic 4: Templates and Conditions

Topic 5: Truth Maintenance and Conflict Resolution

Topic 6: Other Stuff

Facts and Rules

- CLIPS operates by maintaining a list of *facts* and a set of *rules* which operate on them.
- A fact is a piece of information such as **(colour green)** or **(parent_of John Susan)**.
- Facts are created by *asserting* them onto the fact database using the **assert** command.

```
CLIPS> (assert (colour green))  
<Fact-1>
```

The <Fact-1> part is the response from CLIPS to say that a new fact (fact number 1) has been placed on the fact database.

Facts and Rules

- The **(facts)** command will list all current facts.

```
CLIPS> (facts)
```

```
f-0 (initial-fact)
```

```
f-1 (colour green)
```

```
For a total of 2 facts.
```

- Facts may also be *retracted* (removed) from the fact database by using the **retract** command.

```
CLIPS> (retract 1)
```

```
CLIPS> (facts)
```

```
f-0 (initial-fact)
```

```
For a total of 1 fact.
```

Facts and Rules

- Two things to note here:
 1. firstly, to retract a fact you must specify a number (the *fact-index*), not the fact itself, and
 2. secondly, fact-indices are not reused. Once fact 0 has been retracted, the next fact asserted will have the index 2, not 0.

Facts and Rules

- A rule is expressed in the form 'IF something is true THEN do some action'.

```
(defrule duck
  (animal-is duck)
=>
  (assert (sound-is quack))).
```

- The rule consists of three parts.
 1. **(defrule duck)**, simply gives the rule a unique name.
 2. **(animal-is duck)**, is the pattern (the IF part) of the rule and
 3. **(assert (sound-is quack))**, is the action (the THEN part).
- In plain language, this rule means 'if there is a fact **(animal-is duck)** on the fact database, then assert another fact, **(sound-is quack)**, onto the fact database'.

Facts and Rules

1. Clear the system, then type in the rule exactly as printed in the previous slide.
2. Typing **(rules)** will give you a list of rules (just the one, in this case) present in the system.
3. At this point, there are no facts present.
4. Now, type **(assert (animal-is duck))**.
5. Check the fact list - there's one fact.
6. To trigger your rule, type **(run)**.
7. Although nothing appears to happen, if you check the fact list again you'll see that there is a new fact, **(sound-is quack)**, which has been inferred by the rule.
8. This is the power of rule-based programming - the ability to make inferences from data, particularly as the results of one rule can be used as the pattern for another.

Facts and Rules

9. Add the rule below:

```
(defrule is-it-a-duck  
  (animal-has webbed-feet)  
  (animal-has feathers)  
=>  
  (assert (animal-is duck)))
```

10. Then type **(reset)** to clear the facts (the rules will be untouched).

Facts and Rules

11. Using the assert statement to add the facts below:

(animal-has webbed-feet)

(animal-has feathers)

Check the Fact window. How many facts are added?

12. **(run)** the rules.

- The rule **is-it-a-duck** has fired and asserted the fact **(animal-is duck)**.
- This fact has then triggered rule duck, which has asserted the fact **(sound-is quack)**.
- Very powerful systems can be built using this ability to *chain* rules.

Facts and Rules

- Asserting facts is a rather unsatisfactory way of presenting results.
- Type in the first rule again, this time with the multiple actions as shown below:

```
(defrule duck
```

```
    (animal-is duck)
```

```
=>
```

```
    (assert (sound-is quack))
```

```
    (printout t "it's a duck" crlf))
```

- Next time you run the rules, you'll get a message on screen as well as the asserted quack fact.

CLIPS Editor

- It's rather inefficient having to type all your rules in each time you run CLIPS.
- You can write rules in a file and give the file a name.
- Before run the rule, load the file using the 'Load Constructs..' command on the file menu.
- Then, choose Run from the execution menu.
- CLIPS will expect a file with the extension .CLP, and there's a handy editor to help you create them.
- You can't put facts in a .CLP file in the same way as you can from the command prompt, so for now you'll still enter them as before.

Exercises 1:

- Given a list of production rules below, convert it to CLIPS.

Rule 1:

IF The patient has a sore throat
AND We suspect a bacterial infection
THEN We believe the patient has strep throat

Rule 2:

IF The patient's temperature is 40C
THEN The patient has a fever

Rule 3:

IF The patient has been sick over a month
AND The patient has a fever
THEN We suspect a bacterial infection

- Please note that you need to represent “The patient has a sore throat” as “The-patient-has-a-sore-throat” in CLIPS.
- What conclusion will you get if the patient's temperature is 40C, the patient has a sore throat and been sick over a month?

CLIPS tutorials

- The following six short tutorials will step you through most of the basic information you need in order to use the CLIPS expert system development tool. The tutorials were developed by Peter Robinson at the University of Hull in Great Britain.

Topic 1: Facts and Rules

Topic 2: Pattern Matching

Topic 3: Wildcards and Variables

Topic 4: Templates and Conditions

Topic 5: Truth Maintenance and Conflict Resolution

Topic 6: Other Stuff

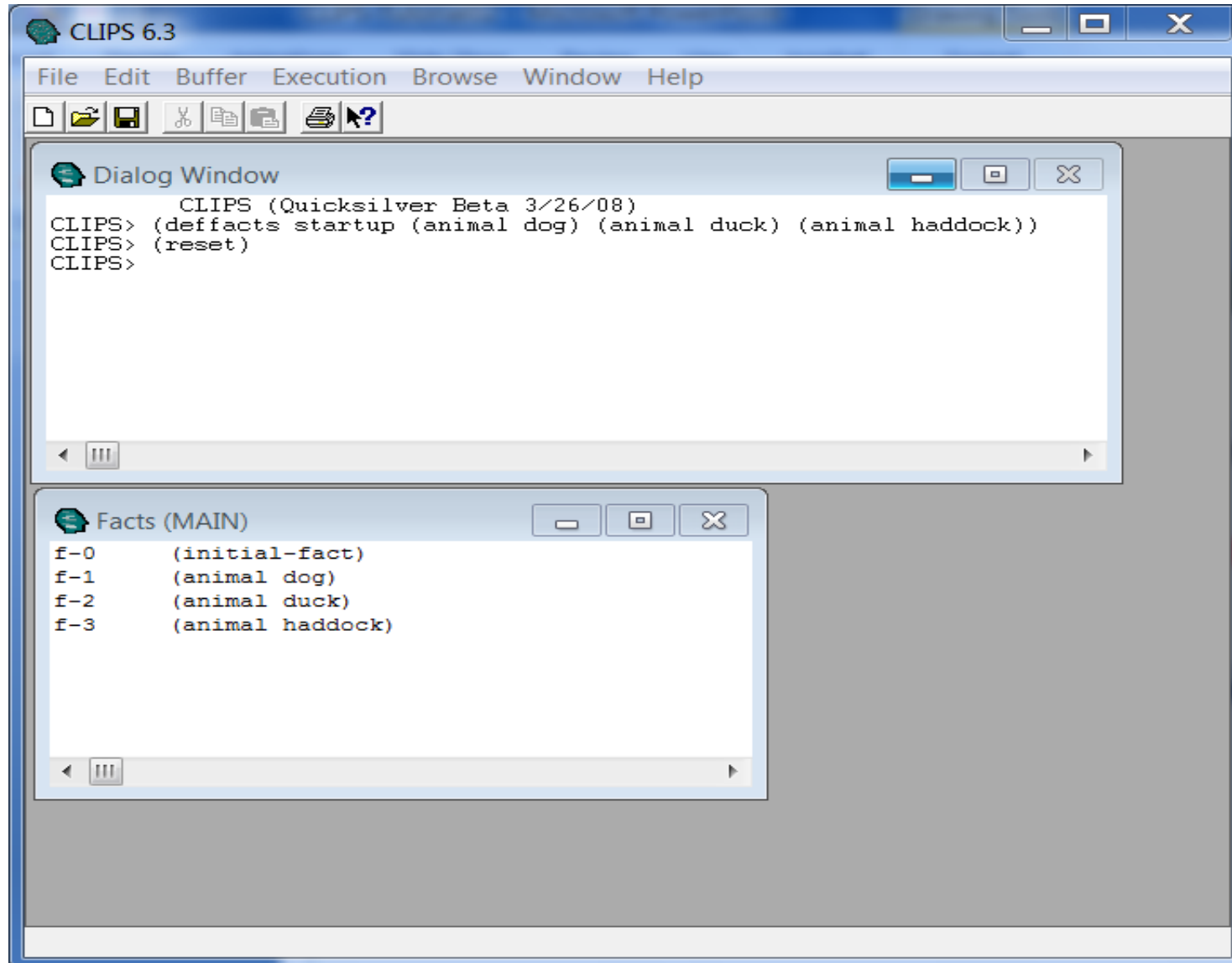
Table of Contents

1. Patterns and Actions
 - i. Defacts and matching things
 - ii. Variables in patterns - wildcards
 - iii. Logic and Math Operators
 - iv. Getting data from users

Deffacts

- In the previous lab, facts are asserted one by one.
- To assert many facts at one time, use the **deffacts** structure.
- For example:
(deffacts startup (animal lion) (animal monkey) (animal bird))
- will assert **three** facts every time CLIPS is reset.

Deffacts



Deffacts: An exercise

- Use the CLIPS editor to enter the following facts in using the **deffacts** statement.
- Then reset CLIPS and look at the fact list to check whether they are present.

(animal dog) (animal cat) (animal duck) (animal turtle)
(warm-blooded dog) (warm-blooded cat)
(warm-blooded duck)
(lays-eggs duck) (lays-eggs turtle)
(child-of dog puppy) (child-of cat kitten) (child-of turtle hatchling)

- How many facts are now available in the fact list?

Matching things

- Previously, the patterns used to match rules against facts are very simple and rather restrictive.
- Each pattern has matched one specific fact.
- By using **wildcards**, it is possible to make rules match multiple facts, executing their actions repeatedly.
- For instance, the rules

```
(defrule animal  
  (animal ?)  
=>  
  (printout t "animal found" crlf))
```

- will produce

```
CLIPS>(run)  
Animal found  
Animal found  
Animal found  
Animal found
```



wildcard

Variable in patterns

- Sometimes we need to have variables where we can refer to them later on in our program.
- Use *?var_name* instead of the ? symbol only.
- For instance, the rule

```
(defrule list-animals  
  (animal ?name)  
=>  
  (printout t ?name " found" crlf))
```

- will produce

```
CLIPS>(run)  
dog found  
cat found  
duck found  
turtle found
```

Multiple pattern matching

- Can do multiple pattern matching
- For instance, the rule

```
(defrule mammal
  (animal ?name)
  (warm-blooded ?name)
  (not (lay-eggs ?name))
=>
  (assert (mammal ?name))
  (printout t ?name " is a mammal" crlf))
```

- will produce

```
CLIPS>(run)
dog is a mammal
cat is a mammal
```

Multiple pattern matching

- Another example:

```
(defrule mammal2
  (mammal ?name)
  (child-of ?name ?young)
=>
  (assert (mammal ?young))
  (printout t ?young " is a mammal" crlf))
```

- will produce

```
CLIPS>(run)
puppy is a mammal
kitten is a mammal
```

Retract facts

- In order to retract a fact, we need to know its fact-index.
- Facts can be retracted from within rules by binding them to variables, like this:

```
(defrule remove-mammal  
    ?fact <- (mammal ?)  
    (child-of ?name ?young)  
  
=>  
    (printout t "retracting " ?fact crlf)  
    (retract ?fact))
```

Logic and Math Operators

- CLIPS uses ***prefix*** notation in its logic and math expressions

```
(defrule take-umbrella
  (or (weather raining)
      (weather snowing))
  =>
  (assert (umbrella required)))
```

- The above rule can be interpreted as “if it is raining or it is snowing” then “take an umbrella”.

```
CLIPS> (+ 5 7)
12
CLIPS> (- 5 7)
-2
```

```
CLIPS> (* 5 7)
35
CLIPS> (/ 5 7)
0.7142857142857143
```

Logic and Math Operators

- Compute the following formula. Represent each formula in prefix notation.

1. $10 + 4 * 5$

2. $4 / 3 * 5$

3. $10 - 2 - 4 + 6$

4. $4 * 6 / 5 + 3 - 6$

Getting data from a user

- CLIPS get information from a user by means of the **(read)** function.
- To demonstrate this, type **(assert (user-input (read)))** and press Enter
- Then type **hello** and press **Enter**.
- What has happen?
- Look at the Fact window, what fact has been inserted?

Getting data from a user

- Another example:

```
(defrule what-is-child
  (animal ?name)
  (not (child-of ?name ?child))
=>
  (printout t "What do you call the child of a " ?name "?")
  (assert (child-of ?name (read))))
```

- Look at the Fact window, what fact has been inserted?

Getting data from a user

- Another example:

```
(defrule ask-sick
  (patient-has-fever)
=>
  (printout t "Question: Does the patient sick over a month?
[yes/no]? " crlf)
  (printout t "Answer [yes/no]: ")
  (assert (patient-sick-over-month(read))))
```

- To run this rule, assert the fact (patient-has-fever) in the fact list.
- Run the rule.
- Look at the Fact window, what fact has been inserted?

Exercise 2:

Using the following rules:

Rule 1:

IF The patient has a sore throat
AND We suspect a bacterial infection
THEN We believe the patient has strep throat

Rule 2:

IF The patient's temperature is 40C
THEN The patient has a fever

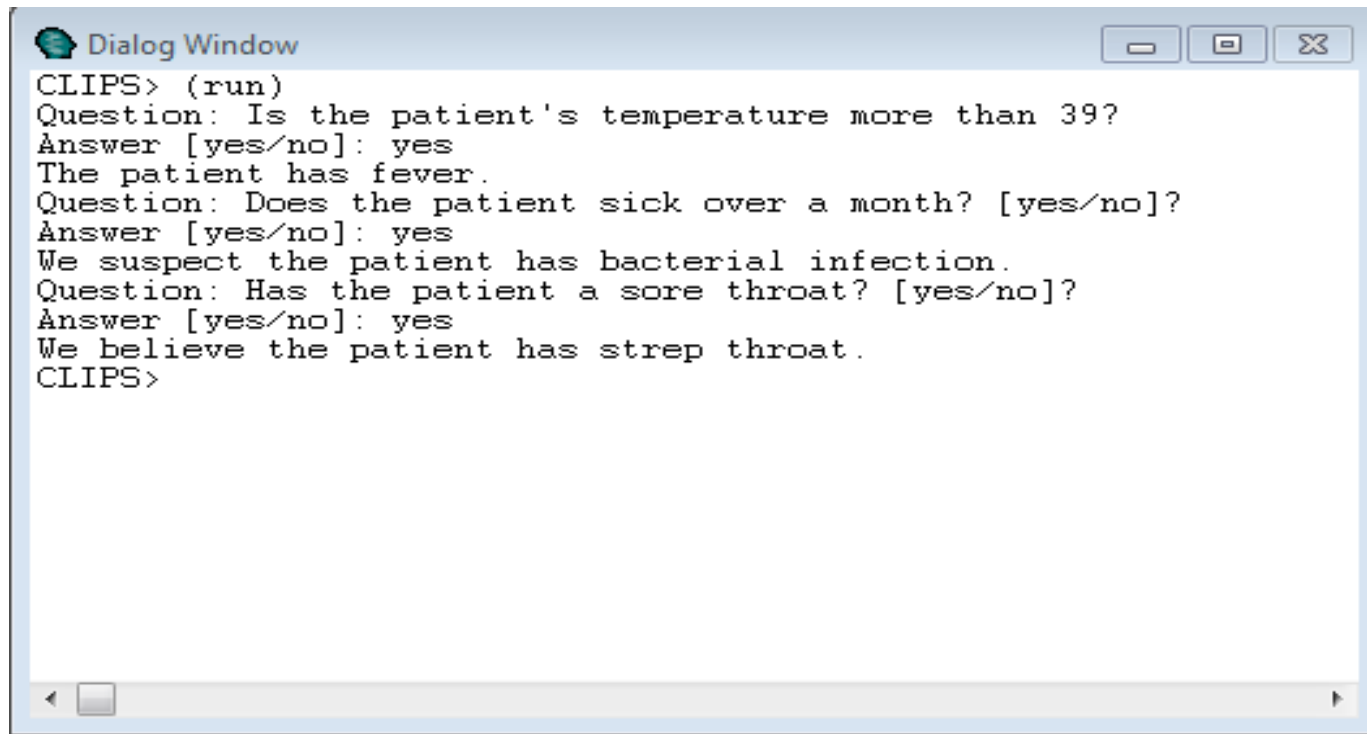
Rule 3:

IF The patient has been sick over a month
AND The patient has a fever
THEN We suspect a bacterial infection

- Develop a question-answer system for the above rule.

Exercise 2 (continue):

- Your sample input/output of your QA system could be look like this.
- Use slide 18 as the reference.
- You need to present the system to me during your next lab session.



```
CLIPS> (run)
Question: Is the patient's temperature more than 39?
Answer [yes/no]: yes
The patient has fever.
Question: Does the patient sick over a month? [yes/no]?
Answer [yes/no]: yes
We suspect the patient has bacterial infection.
Question: Has the patient a sore throat? [yes/no]?
Answer [yes/no]: yes
We believe the patient has strep throat.
CLIPS>
```

CLIPS tutorials

- The following six short tutorials will step you through most of the basic information you need in order to use the CLIPS expert system development tool. The tutorials were developed by Peter Robinson at the University of Hull in Great Britain.

Topic 1: Facts and Rules

Topic 2: Pattern Matching

Topic 3: Wildcards and Variables

Topic 4: Templates and Conditions

Topic 5: Truth Maintenance and Conflict Resolution

Topic 6: Other Stuff

More about wild patterns

- Suppose we have the facts:

```
(member-of beatles john_lennon paul_mccartney george_harrison  
ringo_starr)  
(member-of who roger_daltrey pete_townsend keith_moon)  
(member-of ebtg tracey_thorn ben_watt)
```

- We wish to have a rule which will be triggered by all three of these facts, we can use the multi-field wildcard, \$?, which will match zero or more symbols

```
(defrule bands  
  (member_of ?band $?)  
=>  
  (printout t "there is a band called " ?band crlf))
```

More about wild patterns

- Produces the results:

```
CLIPS>(run)
there is a band called ebtg
there is a band called ebtg who
there is a band called ebtg beatles
```

- We also can get a list of all members of all bands:

```
(defrule band-members
  (member_of ?band $? ?member $?)
=>
  (printout t ?member " is a member of " ?band crlf))
```

- The LHS of this rule, the multi-field wildcard will match zero or more symbols. The wildcard ?member will only match one symbol at a time.

More about wild patterns

- The ?member wild card will match once for each individual member of a band, while the \$? Wildcards match to the other preceding and following members.

Match#	first \$? matches	?member matches	last \$? matches
1	nothing	john_lennon	paul_mccartney george_harrison ringo_starr
2	john_lennon	paul_mccartney	george_harrison ringo_starr
3	john_lennon paul_mccartney	george_harrison	ringo_starr
4	john_lennon paul_mccartney george_harrison	ringo_starr	nothing

- We can also use multi-field wildcard variables:

```
(defrule band-members
  (member_of ?band $?members)
=>
  (printout t "The members of " ?band " are " $?members crlf))
```


More about variables

- So far, we have incorporated a variable in a rule on the LHS of rule.
- Use the ***bind*** function to create a temporary variable in the right hand side of a rule:

```
(defrule addup
  (number ?x)
  (number ?y)

=>
  (bind ?total (+ ?x ?y))
  (printout t ?x " + " ?y " = " ?total crlf)
  (assert (total ?total)))
```

More about variables

- The temporary variable only exists inside the rule which bind it.
- If we need variables that can be used by more than one rule or function without losing their values.

```
(defglobal  
  ?*var1* = 17  
  ?*oranges* = "seven")
```

- After a (reset), there will be two global variables, ?*var1* and ?*oranges* with the values 17 and “seven” respectively, which may be accessed by any rule. To change their values, the bind function can be used.

CLIPS tutorials

- The following six short tutorials will step you through most of the basic information you need in order to use the CLIPS expert system development tool. The tutorials were developed by Peter Robinson at the University of Hull in Great Britain.

Topic 1: Facts and Rules

Topic 2: Pattern Matching

Topic 3: Wildcards and Variables

Topic 4: Templates and Conditions

Topic 5: Truth Maintenance and Conflict Resolution

Topic 6: Other Stuff

Templates

- Some facts that we created are link to each other.
- For example, the following facts are related to each other:
 - (age Johan 21)**
 - (weight Johan 80)**
 - (height Johan 175)**
 - (blood-pressure Johan 130 80)**
- The above facts can be grouped by using the **deftemplate** function.

Template

(deftemplate personal-data

(slot name)

(slot age)

(slot weight)

(slot height)

(multislot blood-pressure))

- **deftemplate** does not any facts, but rather the form which facts can take.
- Every time a fact of this type is created, it contains the slots specified in its definition, each of which can contain a value and can be accessed by name.

Template

- Each person's data can now be asserted thus:

```
(assert (personal-data (name Johan) (age 21)  
  (weight 80) (height 175)  
  (blood-pressure 130 80)) )
```

- Or in a deffacts structure:

```
(deffacts people  
  (personal-data (name Johan) (age 21) (weight 80) (height 175)  
    (blood-pressure 130 80))  
  (personal-data (name Johana) (age 20) (weight 70) (height 160)  
    (blood-pressure 130 90)))
```

Template

- It is not compulsory to have all the values, or insert the values in a certain order, add the following in your deffacts:

(personal-data (weight 110) (age 23) (name Brenda))

Template

- Template facts can be altered without the need to retract them and assert a new version.
- The function `modify` allows you to change the value of one or more slots on a fact.
- Suppose it's Johan's birthday. So we will asserting the fact **(birthday Johan)**. If we define the rule

```
(defrule birthday
  ?birthday <- (birthday ?name)
  ?data-fact <- (personal-data (name ?name) (age ?age))
=>
  (modify ?data-fact (age (+ ?age 1))) (retract ?birthday) )
```

- The rule will modify Johan's age while leaving all his other personal data intact.

Conditional elements in rules

- There are several ways of employing conditional elements in rules.
- The first is the logical and which is implied simply by using two patterns in the left hand side of a rule, thus:

```
(defrule print-ages
  (personal-data (name ?name) (age ?age))
=>
  (printout t ?name " is " ?age " years old." crlf) )
```

- The above rule says “if there is a fact with a name and an age, then print it out”.

Conditional elements in rules

- Second by using the **and/or/not** operator:

```
(defrule print-weight-and-ages
  (and
    (personal-data (name ?name) (age ?age))
    (personal-data (name ?name) (weight ?weight))
  )
=>
  (printout t ?name " weighs " ?weight " at " ?age " years
old." crlf) )
```

Conditional elements in rules

- Second by using the **and/or/not** operator:

```
(defrule print-ages-22-32
  (or
    (personal-data (age 22) (name ?name))
    (personal-data (age 32) (name ?name))
  )
=>
  (printout t ?name crlf)
)
```

Conditional elements in rules

- Second by using the **and/or/not** operator:

```
(defrule not-birthday
  (personal-data (name ?name) (age ?age))
  (not (birthday ?name))
=>
  (printout t "It's not " ?name " 's birthday" crlf) )
```

Conditional elements in rules

- Third by using the **test** function: allows you to check anything in the LHS of a rule.

```
(defrule big-man
  (personal-data (name ?name) (weight ?weight))
  (test (> ?weight 100))
=>
  (printout t ?name " weighs " ?weight " kg." crlf) )
```

Conditional elements in rules

- Fourth by using the **exists/forall** function:
 - exists is satisfied if there are one or more facts which match its predicate.
 - forall – if every facts match its pattern

```
(defrule are-there-people
  (exists (personal-data (name ?name)))
=>
  (printout t "Rule are-there-people reports there is at
least one person" crlf) )
```

Exercises

1. Declare a template of students which consists of name, id num, semester, gpa and cpa.
2. Create several data of students.
3. Print out a list of students who get more than cpa 3.75.
4. Print out who are in Dean's list.

CLIPS tutorials

- The following six short tutorials will step you through most of the basic information you need in order to use the CLIPS expert system development tool. The tutorials were developed by Peter Robinson at the University of Hull in Great Britain.

Topic 1: Facts and Rules

Topic 2: Pattern Matching

Topic 3: Wildcards and Variables

Topic 4: Templates and Conditions

Topic 5: Truth Maintenance and Conflict Resolution

Topic 6: Other Stuff

Truth maintenance

- Before this we have come across several conditional elements: **and**, **or**, **not**, **test**, **exists** and **forall**.
- There is one more: **logical** - assists with the perennial problem of truth maintenance.
- Suppose the following data:

(deftemplate personal-data

(slot name) (slot age) (slot weight)

(slot smoker) (multislot date-of-birth))

Truth maintenance

(deffacts people

 (personal-data (name adam) (weight 60) (age 30)
 (smoker no) (date-of-birth 18 06 1970))

 (personal-data (name brenda) (weight 120) (age 45)
 (smoker yes) (date-of-birth 18 06 1955))

 (personal-data (name charles) (weight 120) (age 60)
 (smoker yes)(date-of-birth 18 06 1940)))

(deffacts data

 (date 18 06 2000))

Truth maintenance

- And the rule below

(defrule cardiac-risk

(personal-data (name ?name) (smoker yes) (weight ?weight))
 (test (> ?weight 100))

=>

(assert (cardiac-risk ?name)))

- Tests whether a person is overweight and a smoker.
- If he is, then it asserts a warning that his heart is at risk.
- What happens if the person gives up smoking, or loses weight?
- The warning fact will still be present, because it is not linked in any way to the rule which created it.
- This means that the fact base is no longer consistent with reality.

Truth maintenance

- We can overcome the problem by using the **logical** element.
- The previous rule can be modified to:

```
(defrule cardiac-risk
```

```
  (logical (personal-data (name ?name) (smoker yes)  
                    (weight ?weight)))
```

```
  (logical (test (> ?weight 100))))
```

```
=>
```

```
  (assert (cardiac-risk ?name)) )
```

- When the above rule is run, the results are identical to those generated by the first version in slide 15 (i.e. the facts **(cardiac-risk brenda)** and **(cardiac-risk charles)** are asserted).

Truth maintenance

- The difference occurs when we change the initial data.
- Make sure the fact window is open, then having run the rule, locate the fact-index of Brenda's personal data (suppose on your system, it's 2) and type at the command line:

(modify 2 (weight 80))

- Now the fact (cardiac-risk brenda) disappears from the fact list.
- By using the **logical** keyword, we have created a link between the fact asserted and the premises on which it was based.
- When the premises changed, their results were no longer valid, so they were removed.

Salience

- One way to control rule firing.
- A salience value is between -10000 and 10000, where 0 is the default value.
- Rules with a higher salience are fired before rules with a lower salience.

Salience

- Consider the following two rules:

```
(defrule poke-fun-at-smokers  
  (personal-data (name ?name) (smoker yes))  
=>  
  (printout t ?name " is a fool." crlf) )
```

```
(defrule worry-about-thin-people  
  (personal-data (name ?name) (weight ?weight))  
  (test (< ?weight 80))  
=>  
  (printout t ?name " is looking a bit thin." crlf) )
```

When you run those rules, then they will both fire, but the order of their firing will depend only upon the order in which the facts on which they depend were created.

Salience

- When we modify the previous two rules into:

```
(defrule poke-fun-at-smokers  
  (declare (salience 10))  
  (personal-data (name ?name) (smoker yes))
```

=>

```
(printout t ?name " is a fool." crlf) )
```

```
(defrule worry-about-thin-people  
  (declare (salience 20))  
  (personal-data (name ?name) (weight ?weight))  
  (test (< ?weight 80))
```

=>

```
(printout t ?name " is looking a bit thin." crlf) )
```

Then the higher salience of rule worry-about-thin-people will ensure that it fires first.

Exercises 1:

- Based on the CLIPS program you develop for the following rules:

Rule 1:

IF The patient has a sore throat
AND We suspect a bacterial infection
THEN We believe the patient has strep throat

Rule 2:

IF The patient's temperature is 40C
THEN The patient has a fever

Rule 3:

IF The patient has been sick over a month
AND The patient has a fever
THEN We suspect a bacterial infection

Exercises 1:

- Add the following two rules in your previous program.
- Reset CLIPS. Assert the facts the patient's temperature is 40C, the patient has a sore throat and been sick over a month. Run the code again.
- What conclusion/s will you get?

Rule 4:

IF The patient has a fever

THEN The patient cannot go to a class

Rule 5:

IF The patient cannot go to a class

THEN The patient should stay home and read a book

Exercises 2:

- Develop a small expert system using CLIPS to diagnose a car's problem. Use your own creativity to design your system input and output.

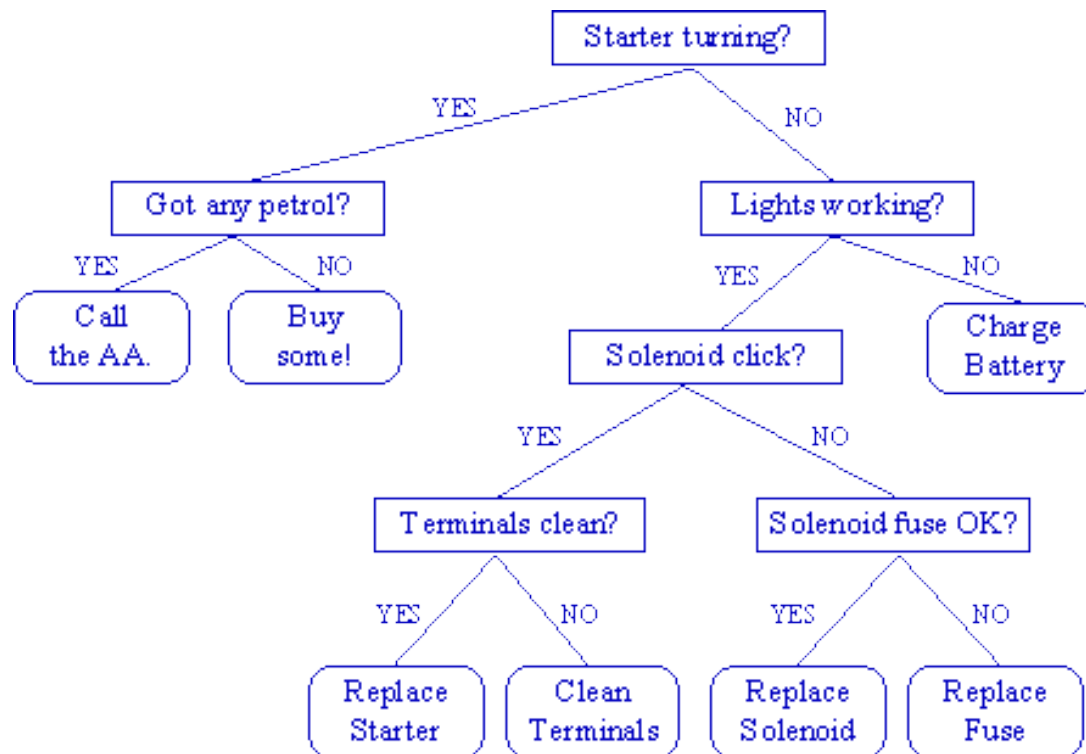


Table of Contents

1. Predicate functions
2. String functions
3. Procedural functions
4. Deffunctions
5. Exercises

CLIPS tutorials

- The following six short tutorials will step you through most of the basic information you need in order to use the CLIPS expert system development tool. The tutorials were developed by Peter Robinson at the University of Hull in Great Britain.

Topic 1: Facts and Rules

Topic 2: Pattern Matching

Topic 3: Wildcards and Variables

Topic 4: Templates and Conditions

Topic 5: Truth Maintenance and Conflict Resolution

Topic 6: Other Stuff

Predicate Functions

Functions	
<code>numberp,</code> <code>floatp,</code> <code>integerp</code>	returns the symbol TRUE if its argument is a float or integer, otherwise it returns the symbol FALSE (integerp <expression>)
<code>lexemep,</code> <code>stringp,</code> <code>symbolp</code>	returns the symbol TRUE if its argument is a string or symbol, otherwise it returns the symbol FALSE. (lexemep <expression>)
<code>evenp,</code> <code>oddp</code>	returns the symbol TRUE if its argument is an even/odd number, otherwise it returns the symbol FALSE.
<code>eq,</code> <code>neq</code> <code>=,</code> <code><></code> (for numeric values)	Comparing equality (eq <expression> <expression>+)
<code>>,</code> <code>>=,</code> <code><,</code>	Comparing values

String Functions

Functions	
<code>(str-cat <expression>*)</code>	concatenates its arguments into a single string.
<code>(sub-string <integer-expression> <integer-expression> <string-expression>)</code>	retrieve a portion of a string from another string.
<code>(str-index <lexeme-expression> <lexeme-expression>)</code>	will return the position of a string inside another string
<code>(upcase <string-or-symbol-expression>)</code> <code>(lowcase <string-or-symbol-expression>)</code>	will return a string or symbol with upper/lowercase alphabetic characters
<code>(str-compare <string-or-symbol-expression> <string-or-symbol-expression>)</code>	Comparing values

Procedural Functions – if...then

defrule can be simplified using **if...then**

using 2 defrules

```
(defrule closed-valves-number-6
  (temp high)
  (valve 6 closed)
=>
  (printout t "The special valve 6 is
              closed!" crlf)
  (assert (perform special
              operation)))

(defrule closed-valves-other-than-6
  (temp high)
  (valve ?v&~6 closed)
=>
  (printout t "Valve " ?v " is normally
              closed" crlf))
```

using **if...then**

```
(defrule closed-valves
  (temp high)
  (valve ?v closed)
=>
  (if
   (= ?v 6)
  then
    (printout t "The special valve " ?v "
                is closed!" crlf)
    (assert (perform special operation))
  else
    (printout t "Valve " ?v " is normally
                closed" crlf)))
```


Procedural Functions – while

Syntax

```
(while <expression> [do]  
  <action>*)
```

Example

```
(defrule open-valves  
  (valves-open-through ?v)  
  =>  
  (while (> ?v 0)  
    (printout t "Valve " ?v " is open" crlf)  
    (bind ?v (- ?v 1))))
```

Procedural Functions – loop-for-count

Syntax

```
(loop-for-count <range-spec> [do] <action>*)
```

```
<range-spec> ::= <end-index> |  
               (<loop-variable> [<start-index> <end-index>])
```

```
<start-index> ::= <integer-expression>
```

```
<end-index>   ::= <integer-expression>
```

Example 1

```
CLIPS> (loop-for-count 2 (printout t "Hello world"  
crLf))  
Hello world  
Hello world
```

Example 2

```
(loop-for-count (?cnt1 2 4) do  
  (loop-for-count (?cnt2 1 3) do  
    (printout t ?cnt1 " " ?cnt2 crLf)))
```

Output

```
2 1  
2 2  
2 3  
3 1  
3 2  
3 3  
4 1  
4 2  
4 3
```

Procedural Functions – switch

Syntax

```
(switch <test-expression>
  <case-statement>
  <case-statement>+
  [<default-statement>])

<case-statement> ::=
  (case <comparison-expression>
  then <action>*)

<default-statement> ::= (default
  <action>*)
```

Example

```
CLIPS> (defglobal ?*x* = 0)
CLIPS> (defglobal ?*y* = 1)
CLIPS>
(deffunction foo (?val)
  (switch ?val
    (case ?*x* then *x*)
    (case ?*y* then *y*)
    (default none)))
CLIPS> (foo 0)
*x*
CLIPS> (foo 1)
*y*
CLIPS> (foo 2)
none
```

Deffunction

- User-defined function
- Comprised of five elements:
 - 1) a name,
 - 2) an optional comment,
 - 3) a list of zero or more required parameters,
 - 4) an optional wildcard parameter to handle a variable number of arguments and
 - 5) a sequence of actions, or expressions, which will be executed in order when the deffunction is called.

Deffunction

Syntax

```
(deffunction <name> [<comment>] (<regular-parameter>* [<wildcard-parameter>])  
  <action>*)
```

<regular-parameter> ::= <single-field-variable>

<wildcard-parameter> ::= <multifield-variable>

Example

```
(deffunction print-args (?a ?b $?c)  
  (printout t ?a " " ?b " and " (length ?c) " extras: " ?c crlf))
```

Function call & Output

```
CLIPS> (print-args 1 2)  
1 2 and 0 extras: ()  
CLIPS> (print-args a b c d)  
a b and 2 extras: (c d)
```

Exercise

Do the following in function:

1. Prompt a number from user, NUM, then calculate the sum of the number from 0 until NUM. The input need to be checked whether it is a number or not to proceed with the operations. Sample output:

```
CLIPS> (totalNumber)
```

```
Insert a number
```

```
D
```

```
Not a number
```

```
CLIPS> (totalNumber)
```

```
Insert a number
```

```
5
```

```
Total 15
```

Exercise

Do the following in function:

2. Read user's name and the user's mark. Print PASS if the marks is above 60, and FAIL otherwise. The name of the user should be printed in capital letters.

```
CLIPS> (marks)
Insert your name
ali
Enter your mark
55
Name: ALI
Grade: FAIL
```

```
CLIPS> (marks)
Insert your name
abu
Enter your mark
78
Name: ABU
Grade: PASS
```